

Homework 2: Image Enhancement & Image Restoration

Hsiang, Wang
313551127

November 14, 2024

1 Implementation

A. Image Enhancement - Gamma correction

The first part of image enhancement is gamma correction; the main idea is to use a strictly increasing function - gamma to adjust the brightness of the image. Set a gamma value such that when $\gamma < 1$, the image will become brighter, and when $\gamma > 1$, the image will become darker. First, normalize each pixel to the range $[0, 1]$ to avoid overflow, apply the gamma exponentiation, and then scale it back to the range $[0, 255]$.

```
TODO Part 1: Gamma correction
"""
Tabnine | Edit | Test | Explain | Document | Ask
def gamma_correction(input_img, gamma):
    output_img = np.power(input_img / 255.0, gamma) * 255.0
    return output_img.astype(np.uint8)
"""
```

B. Image Enhancement - Histogram equalization

In the histogram equalization method, since we only want to adjust the brightness of the image, so we first convert our RGB format to HSV format by using **cvtColor** function from cv2. Then we take out the v-channel, which controls the brightness of the image. After that, the **histogram** function from NumPy is used to measure the histogram of each pixel value in the bins in the range $[0, 255]$. CDF is calculated by using **cumsum** and shifting it to get a mapping of the cdf from the v-channel value. Apply mapping to each pixel so we can get an equalized v-channel to insert back into the HSV image.

```

"""
TODO Part 2: Histogram equalization
"""
Tabnine | Edit | Test | Explain | Document | Ask
def histogram_equalization(input_img):
    hsv_image = cv2.cvtColor(input_img, cv2.COLOR_BGR2HSV)
    v_channel = hsv_image[:, :, 2]
    hist, bins = np.histogram(v_channel.flatten(), bins=256, range=[0,256], density=True)
    cdf = hist.cumsum()
    cdf -= cdf.min()
    cdf = (255 * cdf).astype('uint8')
    equalized_v = cdf[v_channel]
    hsv_image[:, :, 2] = equalized_v
    output_img = cv2.cvtColor(hsv_image, cv2.COLOR_HSV2BGR)
    return output_img

```

C. Image Enhancement - Bonus - CLAHE (Contrast Limited Adaptive Histogram Equalization)

In the CLAHE method, we also convert the image to the HSV format, and then we split our image into many tiles based on grid size. The process is similar to histogram equalization, but we process one tile at a time instead of processing the entire image. After computing the histogram for each tile, we use a clipping method to remove values that exceed the preset clip limit and then evenly distribute the clipped portion across the bins. This way, we obtain a size-limited mapping, and by applying this mapping to each tile and stitching them back together, we can reconstruct the original-sized image.

```

def other_enhancement_algorithm(input_img, cliplimit = 7.0, gridsize = (8, 8)):
    hsv_image = cv2.cvtColor(input_img, cv2.COLOR_BGR2HSV)
    v_channel = hsv_image[:, :, 2]
    h, w = v_channel.shape
    h_tile_size, w_tile_size = h // gridsize[0], w // gridsize[1]
    output_tiles = np.zeros((gridsize[0], gridsize[1], h_tile_size, w_tile_size), dtype=np.uint8)
    output_v_channel = np.zeros_like(v_channel, dtype=np.uint8)
    for i in range(gridsize[0]):
        for j in range(gridsize[1]):
            h_start, h_end = i * h_tile_size, (i + 1) * h_tile_size
            w_start, w_end = j * w_tile_size, (j + 1) * w_tile_size
            tile = v_channel[h_start:h_end, w_start:w_end]
            hist, bins = np.histogram(tile.flatten(), bins=256, range=[0,256])
            clip_value = int(cliplimit * tile.size / 256)
            clipped_hist = np.clip(hist, 0, clip_value)
            excess_sum = hist.sum() - clipped_hist.sum()
            add_back = excess_sum // 256
            clipped_hist += add_back
            cdf = clipped_hist.cumsum()
            cdf = 255 * (cdf - cdf.min()) / (cdf.max() - cdf.min())
            cdf = cdf.astype('uint8')
            output_v_channel[i * h_tile_size: (i + 1) * h_tile_size, j * w_tile_size: (j + 1) * w_tile_size] = cdf[tile].reshape(h_tile_size, w_tile_size)
    hsv_image[:, :, 2] = output_v_channel
    output_img = cv2.cvtColor(hsv_image, cv2.COLOR_HSV2BGR)
    return output_img

```

D. Image Restoration - Motion blur PSF generation

In the PSF kernel I set, I use length as the size of the kernel and set the object displacement value to length. Then, I calculate the points at a length distance from the center using the given angle. With these two points, I use cv2.line to draw a line with a thickness of 1. Finally, after normalization, the PSF function is obtained.

```

"""
TODO Part 1: Motion blur PSF generation
"""
Tabnine | Edit | Test | Explain | Document | Ask
def generate_motion_blur_psf(length, angle, img):
    psf = np.zeros((length, length), dtype=np.float32)
    center = (psf.shape[0] // 2, psf.shape[1] // 2)
    end = (int(center[0] - length * np.cos(np.deg2rad(angle)) ),
           int(center[1] + length * np.sin(np.deg2rad(angle)) ))
    cv2.line(psf, center, end, (255,), thickness=1)
    psf /= psf.sum()
    return psf
"""

```

To ensure that all threads start executing simultaneously, I used **pthread_barrier_t** to allow a specific number of threads to wait for each other before proceeding. The **pthread_barrier_wait** function controls the timing for simultaneous execution. Finally, **pthread_join** is used to wait for all threads to complete, and then the barrier is destroyed, completing the work in the main thread.

E. Image Restoration - Wiener filtering

Since Wiener filtering is usually applied to grayscale images, I apply it separately to the three channels. The process involves performing an FFT on each image channel to obtain G , then applying the previously defined PSF to perform an FFT to get H . With G and H , we can use the formula from the class (shown below) to compute the output. It is important to note that the variable K in the formula represents the noise level, which I have set, and I use the `conj` function to calculate the conjugate of H . This avoids the need to divide by H again, as H may contain very small values that could lead to division issues. Finally, using `ifft`, we can do an inverse Fourier transfer back to the time domain, which is the desired output form.

$$\hat{F}(u,v) = \left[\frac{1}{H(u,v)} \frac{|H(u,v)|^2}{|H(u,v)|^2 + K} \right] G(u,v)$$

Figure 1: Formula of Wiener filtering

```

TODO Part 2: Wiener filtering
"""
Tabnine | Edit | Test | Explain | Document | Ask
def wiener_filtering(img, psf, K):
    out_img = np.zeros_like(img)
    for i in range(3):
        channel = img[:, :, i]
        G = np.fft.fft2(channel)
        H = np.fft.fft2(psf, channel.shape)
        wiener_filter = np.conj(H) / (np.abs(H)**2 + K)
        f_channel = wiener_filter * G
        out_channel = np.fft.ifft2(f_channel)
        out_channel = np.abs(out_channel).astype(np.float32)
        out_channel = np.clip(out_channel, 0, 255)
        out_channel = out_channel.astype(np.uint8)
        out_img[:, :, i] = out_channel
    return out_img

```

F. Image Restoration - Constrained least squares filtering

The process of Constrained Least Squares Filtering is similar to Wiener filtering. Both involve performing FFT on the image channels to obtain G , and applying FFT to the PSF to get H . The only difference is that K is replaced with $\gamma * \text{the FFT of } P$, where P is shown below. We adjust the noise level by setting the value of γ . The subsequent steps are the same—after performing the inverse FFT, the result is inserted back into the original image.

$$\hat{F}(u, v) = \left[\frac{H^*(u, v)}{|H(u, v)|^2 + \gamma |P(u, v)|^2} \right] G(u, v)$$

Figure 2: Formula of Constrained least squares filtering

$$p(x, y) = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Figure 3: Array P

```

"""
TODO Part 3: Constrained least squares filtering
"""
Tabnine | Edit | Test | Explain | Document | Ask
def constrained_least_square_filtering(img, psf, gamma):
    out_img = np.zeros_like(img)
    P = np.fft.fft2([[0, -1, 0],
                    [-1, 4, -1],
                    [0, -1, 0]],
                    out_img[:, :, 0].shape)
    for i in range(3):
        channel = img[:, :, i]
        G = np.fft.fft2(channel)
        H = np.fft.fft2(psf, channel.shape)
        cls_filter = np.conj(H) / (np.abs(H)**2 + gamma * np.abs(P)**2)
        f_channel = cls_filter * G
        out_channel = np.fft.ifft2(f_channel)
        out_channel = np.abs(out_channel).astype(np.float32)
        out_channel = np.clip(out_channel, 0, 255)
        out_channel = out_channel.astype(np.uint8)
        out_img[:, :, i] = out_channel
    return out_img

```

2 Results & Analysis

A. Task 1: Image Enhancement

Gamma Correction:

Gamma = 0.2

Gamma = 0.45

Gamma = 1.5



We can observe that the gamma value decreases, especially when the gamma is less than 1; the corrected image becomes brighter. At gamma = 0.2, the image becomes overly bright, resulting in a strange color appearance and too bright outdoor lighting.

At $\gamma = 0.45$, the result is more balanced, allowing the details of the darker areas of the original image to be visible while the outdoor lighting is not overly bright. At $\gamma = 1.5$, when γ exceeds 1, the image becomes darker, and the outdoor lighting appears noticeably dimmer.

Histogram Equalization:



Compare the above two methods:

After comparing the results in the above figure, we found that gamma does not perform as well in the darker areas as Histogram Equalization, especially in regions with less contrast. Additionally, gamma requires manual tuning, and its value significantly impacts the result. Histogram Equalization does not require setting a gamma value, making it a more general-purpose choice. However, as seen in the result images, Equalization can make the lighting in some areas look unnatural. Furthermore, we can observe that the noise is amplified because Equalization also increases the noise, particularly in areas with low contrast. However, it can also be observed that in Histogram Equalization, the originally bright outdoor lighting becomes too bright, resulting in a

loss of details. This happens because we use global pixels as the target. In the original image, dark pixels dominate, so the bright areas become even brighter during Equalization, leading to losing details. However, there is no such problem in bright areas in gamma correlation.

Bonus:

I implemented CLAHE (Contrast Limited Adaptive Histogram Equalization) for the bonus part. The main concept of CLAHE is to address the issues in Histogram Equalization, where using global pixels for equalization can enhance noise and cause overexposure or underexposure. CLAHE divides the image into multiple tiles, so each tile only uses local pixels for processing. It also uses a clip limit to restrict excessive pixel values, distributing them evenly across the pixels in each tile.



From the images, we can see that CLAHE performs better than gamma correction in areas with low contrast while also addressing the distortion issue in over-bright regions often seen with Histogram Equalization. CLAHE effectively reduces noise as well. Although the tile seams are noticeable in my displayed image, this can be resolved by allowing each tile to overlap, using weights to blend the overlapping areas, or by applying interpolation to smooth it.

B. Task 2: Image Restoration

Minimum Mean Square Error (Wiener) Filtering:

Length = 55, Angle = 45, $K = 0.007$:



Figure 4: test case 1

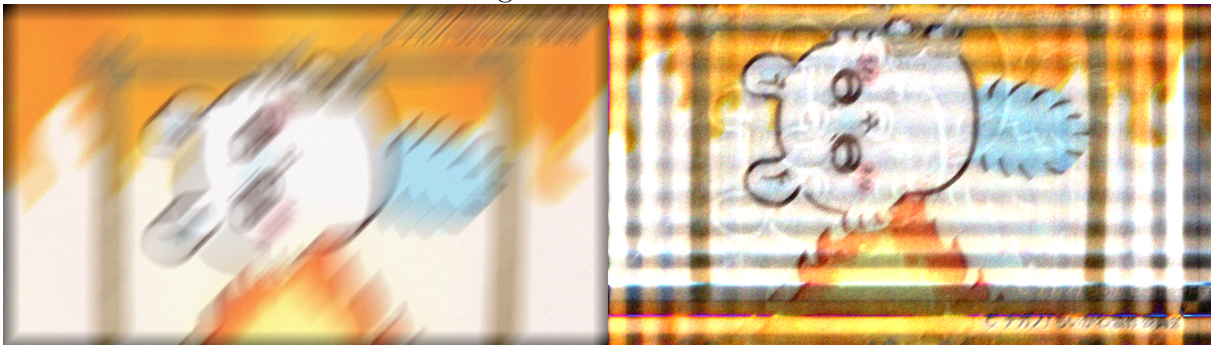


Figure 5: test case 2

Constrained Least Squares Restoration:

Length = 55, Angle = 45, $\gamma = 3.5$:



Figure 6: test case 1

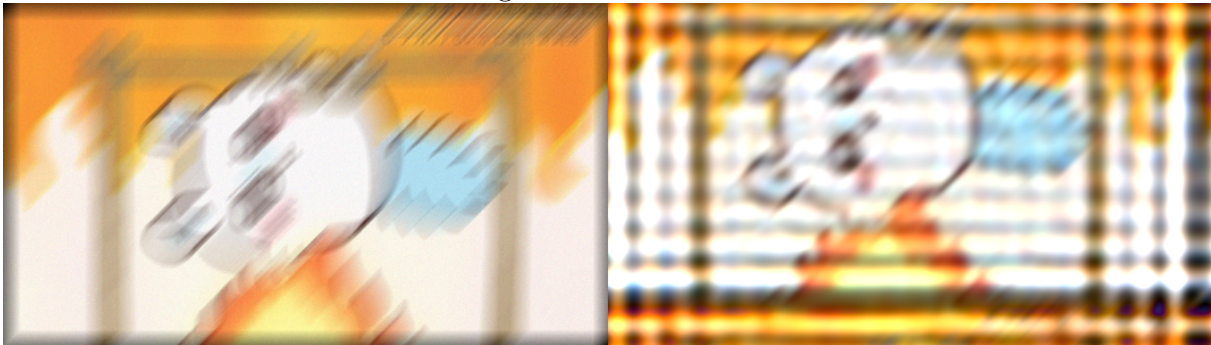


Figure 7: test case 2

Compare the above two methods on different test cases

In the first test case, both methods allow the license plate number to be visible, but the Wiener filter reveals the small white text below the plate number, making it clearer. Overall, CLS is slightly more blurred compared to the Wiener filter. However, since the original image underwent Gaussian blur, both outputs are affected by noise. Nevertheless, CLS produces less noise overall, which can also be observed through a higher PSNR for CLS.

The results in the second test case are similar to those in the first; the Wiener filter shows better clarity, but CLS is more effective at noise reduction. For example, the rope tied to the flying squirrel's paw is clearer in the Wiener filter result, while the noise is lesser in the background of CLS.

However, both methods have the same drawbacks, which can be divided into two issues. First, there is a shift in the image. This occurs because, during FFT, we combine multiple sections of the original image and then apply the transformation. Since we are applying motion blur, the kernel causes the image to shift. Second, many line-like noises appear in the image due to noise; when we perform convolution, the noise is also calculated and amplified.

Bonus:

3 Answer the questions

1. Please describe a problem you encountered and how you solved it.

When applying Wiener filtering, it is typically done on grayscale images, but our input is RGB. Initially, I applied it directly to the image, but the result looked strange. Later, I resolved this by separating the three channels and performing FFT on each channel individually.

2. What potential limitations might arise when using Minimum Mean Square Error (Wiener) Filtering for image restorations? Please suggest possible solutions to address them.

A limitation of Wiener filtering is that we must have some prior knowledge about the filter H we are processing. For example, in this assignment, we need to know that it is motion blur and estimate the length and angle before applying Wiener filtering. Additionally, Wiener filtering does not handle noise particularly well and is prone to being influenced by noise, which can degrade the output quality.

One way to improve this is by using statistical models to help us acquire prior knowledge about the filter H , allowing Wiener filtering to bypass the limitation of needing to know the PSF. For noise reduction, we can use wavelet-based Wiener filtering to minimize the impact of noise on the output.

3. What potential limitations might arise when using Constrained Least Squares Restoration for image restorations? Please suggest possible solutions to address them.

In the CFS method, we can use the optimization techniques taught in class to update our gamma. The problem is that to update gamma, we need η , or we must set η , and different values of η can significantly impact the result. Moreover, the direction and length are fixed in the optimization process, which can lead to bad optimization.

To solve this, in the optimization part, we can set adaptive regularization to allow for more flexible regularization. As for the parameter settings, we can use deep learning methods to find better parameters.